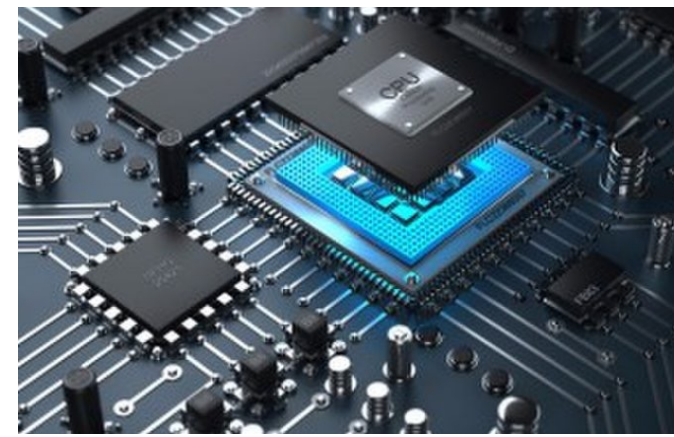
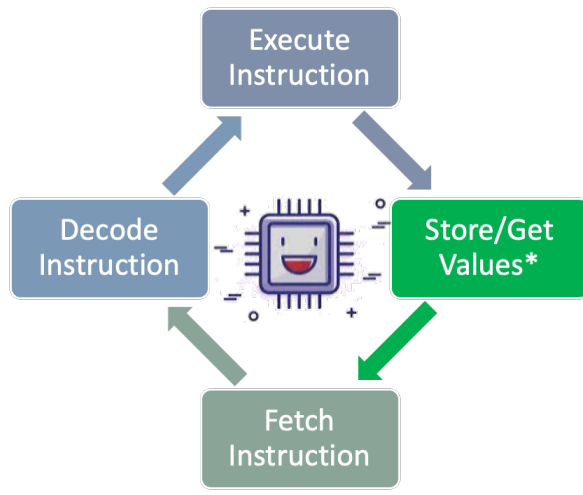




Lecture 04: CPU Organization Fundamentals

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Administrative

New Homework due on Monday!

Lab on Friday!!

For Friday section: Watch the video “**Lecture 04B VIDEO**” (it’s on Canvas)

You will also need it for your new hwk!!!

- **CS64_Lecture04B_syscall_VIDEO.mp4**

Read ALL of Handout 2 (on Canvas) by next lecture

A worksheet file is available for today’s lecture on Canvas!

REMINDER: Come to our office hours!! 😊

So... Like...

What Processes Stuff In A Computer?



The Central Processing Unit (CPU)

- Executes program instructions

Typical capabilities of CPU include:

- Add
- Subtract
- Multiply
- Divide
- Move data from location to location

***You can do just about anything
with a computer with only these
simple instructions!***

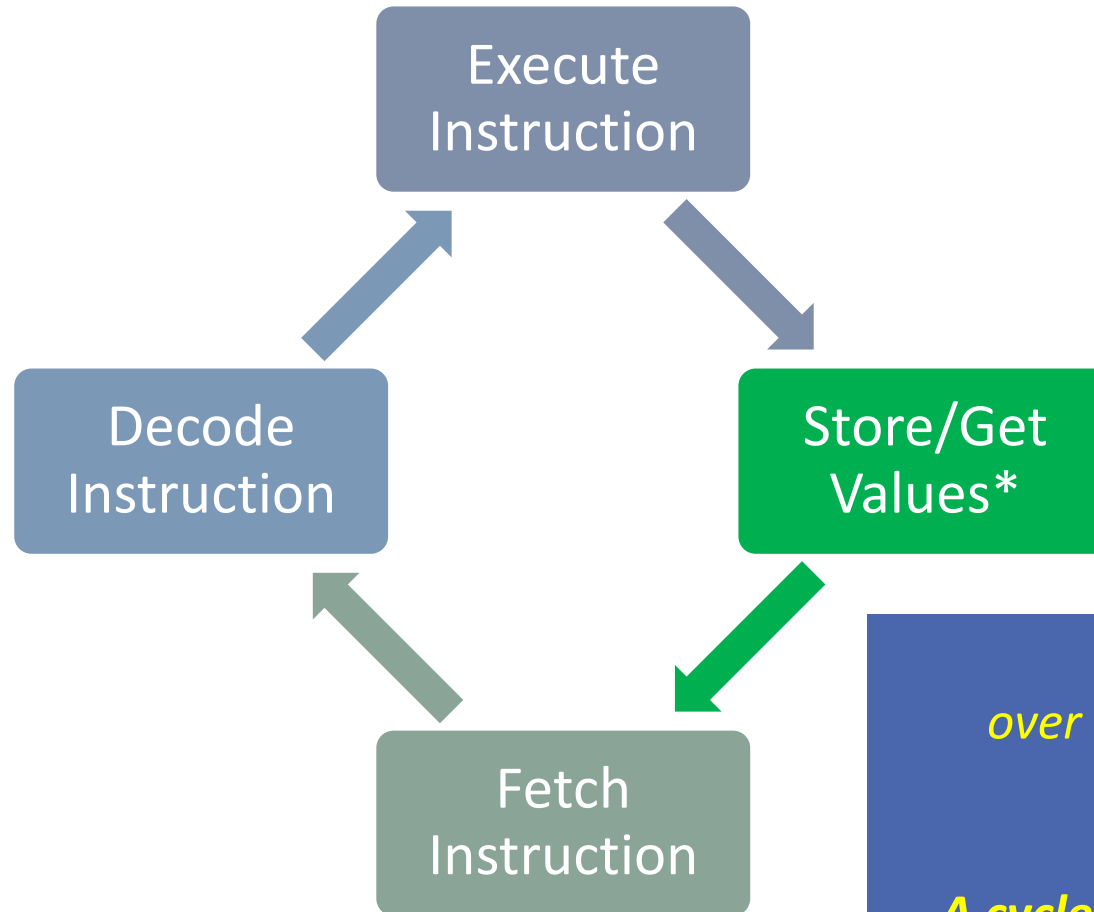
That Digital Machine

What are actual operations a CPU can do?



AND THAT'S *almost* "ALL" YOU NEED A CPU TO DO!!!

The CPU Fetch-Execute Cycle



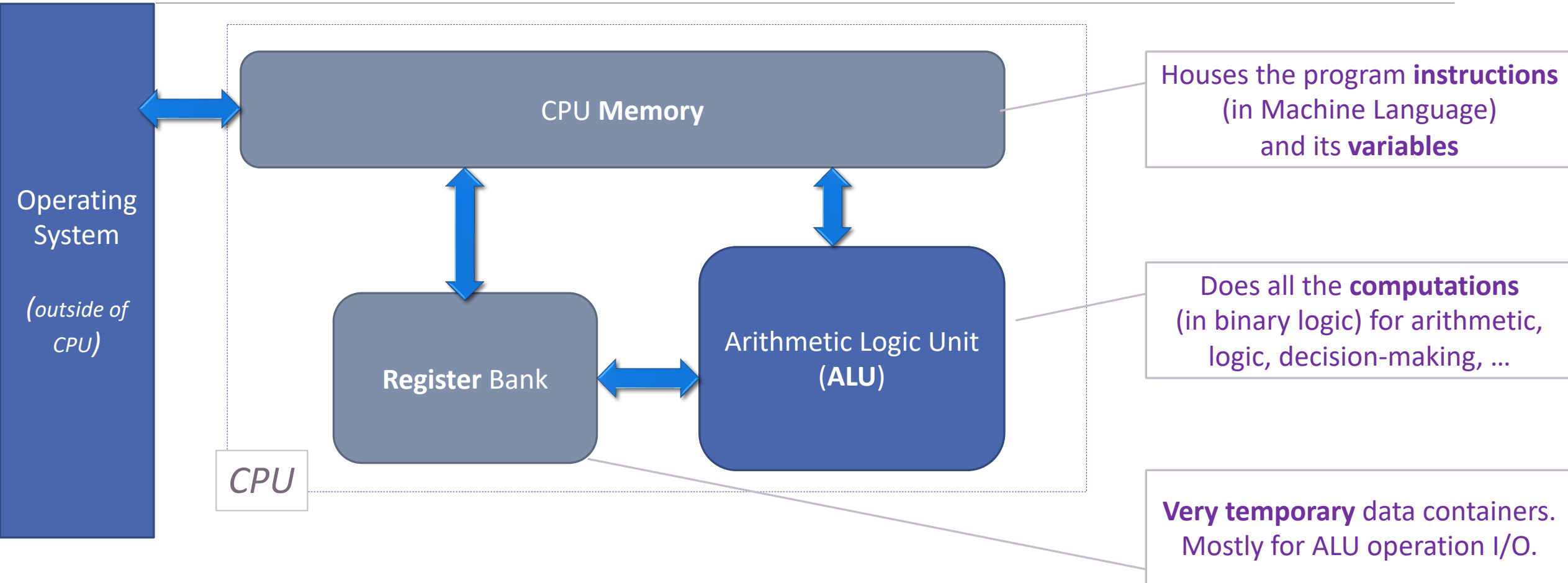
** in memory, not in registers*

A CPU does this cycle over and over and over and... again.

And VERY fast too!
A cycle can take just a few nano-seconds

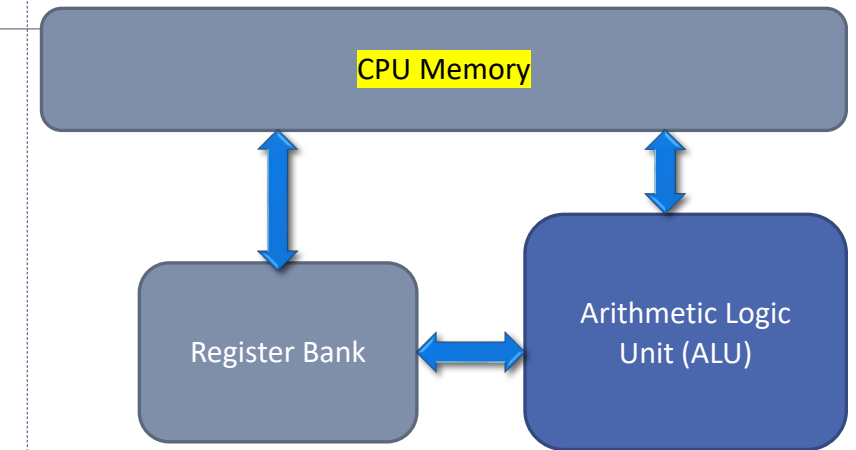
nano = 10^{-9} , or 1 billionth

Simplified View of a CPU



Simplified / Generalized CPU Memory Map

Address	Values
Addresses Range 1*	Reserved for OS (Kernel memory)
Addresses Range 2*	User Variables (stack, heap, static vars)
Addresses Range 3*	User Program Instructions
Addresses Range 4*	Reserved for OS (Kernel memory)



** More about what these “ranges” are in an upcoming lecture...*

Core Components of a CPU

What we need in a CPU is:

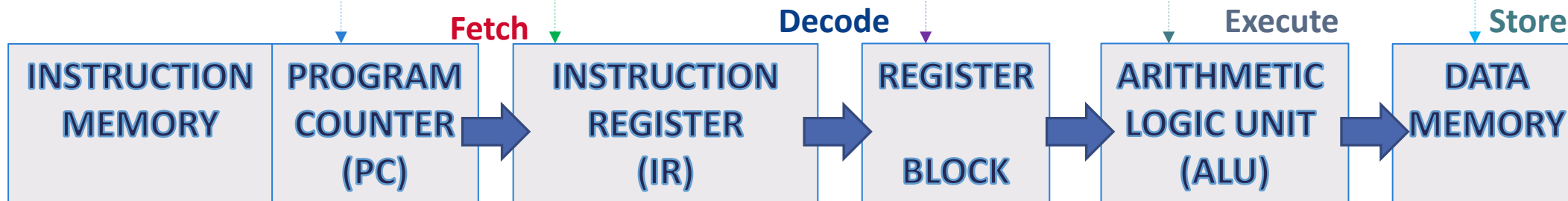
Some *place* to tell us *which instruction is next* (i.e. the *address* of the next instruction)

Some *place* to hold the *current instruction*

Some *place* to hold the program *variables-that-we-need-to-operate-on-now* (short term)

Some *place* to do arithmetic or logic operations on the *variables*

Some *place* to keep the program variables as we're *running the program* (i.e. *stack/heap/static*)



Machine vs. Assembly Language

Machine language (ML) is the actual 1s and 0s

Example, here's 32 bits of machine language:

10111101110111000001010101010000

Assembly language is one step of abstraction “above” ML

- Instructions are given mnemonic codes but still displayed one step at a time
- Advantage? Better human readability

Example:

```
lw    $t0, 4($sp)    # fetch N from someplace in memory
add   $t0, $t0, $t0   # add N to itself and store the result in N
```

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
  muli $2, $5, 4
  add  $2, $4, $2
  lw   $15, 0($2)
  lw   $16, 4($2)
  sw   $16, 0($2)
  sw   $15, 4($2)
  jr   $31
```

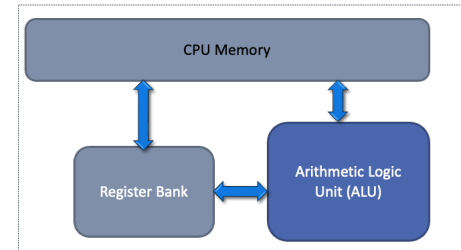
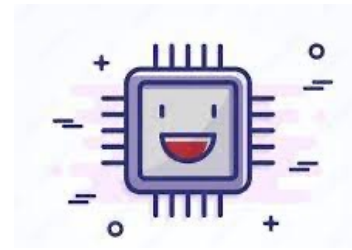
Assembler

Binary machine
language
program
(for MIPS)

```
00000000101000010000000000011000
000000000000110000001100000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
0000001111100000000000000001000
```



Wherefore MIPS ?!



MIPS is a commercially available CPU

- Was designed about 40 years ago and is still in use today!!
- Mostly used today in some “embedded systems”, not in modern desktop/laptop computers

The *architecture* of the MIPS CPU is called **RISC** (Reduced Instruction Set Computer)

- It is relatively simpler than the other major type of CPU architecture (**CISC** : e.g., what Intel/AMD use)
- RISC CPUs are also used heavily in GPUs (e.g. Nvidia) and low-power cores (e.g. ARM in iPhones)
- The concepts and main features in MIPS are same/similar in all modern CPUs
- The instructions for MIPS are easier to learn too
- Popular architecture to teach this course with

The Simple Language of a MIPS CPU

Restrictions:

Can only assign **integers** directly to variables (for now...)

Can only do arithmetic on (e.g. add) variables, ***always two at a time*** (no more)

EXAMPLE:

$$z = 5 + 7$$

has to be simplified to:

Assign **5** to some reg. **x** (i.e. **x = 5**)

Assign **7** to some reg. **y** (i.e. **y = 7**)

Add **x** to **y** and assign answer to reg. **z** (**z = x + y**)

Note: 32 bits!

0000 0000 0000 0000 0000 0000 0000 0101

0000 0000 0000 0000 0000 0000 0000 0111

0000 0000 0000 0000 0000 0000 0000 1100

Note: These 32 bit “containers” can hold on to these binary values in a CPU. They’re called “**REGISTERS**”.

The Simple Language of a MIPS CPU

Assign **5** to **x** (i.e. **$x = 5$**)

Assign **7** to **y** (i.e. **$y = 7$**)

Add **x** to **y** and assign answer to **z** (**$z = x + y$**)

BUT! You can't just give the CPU this high-level jargon!

CPUs only understand binary messages (***instructions***)

aka machine language

Instruction Register

?

Registers

x: ?

y: ?

z: ?

Program Counter

?

Memory

?

Arithmetic Logic Unit

?

Has the **ACTUAL** instruction in it, in M.L.

Instruction Register

x = 5;

Registers

x: 5

y: ?

z: ?

Just a list of the registers
(they aren't really called x, y, and z)

Has the **ACTUAL** address of the **next** instruction in here

Program Counter

0

Note that program line numbers do **not** correspond exactly to the **address** of the instructions in memory. We will discuss this more in later lectures.

Has **ALL** the instructions here, i.e. the program.

Memory

0: x = 5;

1: y = 7;

2: z = x + y;

Arithmetic Logic Unit

?

Has the **ACTUAL** instruction in it, in M.L.

Instruction Register

x = 5;

Registers

x: 5

y: 7

z: ?

Just a list of the registers
(they aren't really called x, y, and z)

Has the **ACTUAL** address of the **next** instruction in here

Program Counter

0 + 1 = 1

Arithmetic Logic Unit

?

Memory

0: x = 5;

1: y = 7;

2: z = x + y;

Has the **ACTUAL** instruction in it, in M.L.

Instruction Register

$z = x + y;$

Registers

x: 5

y: 7

z: ?

Just a list of the registers
(they aren't really called x, y, and z)

Has the **ACTUAL** address of the **next** instruction in here

Program Counter

$1 + 1 = 2$

Arithmetic Logic Unit

?

Memory

0: x = 5;

1: y = 7;

2: z = x + y;

Has the **ACTUAL** instruction in it, in M.L.

Instruction Register

$z = x + y;$

Memory

0: $x = 5;$
1: $y = 7;$
2: $z = x + y;$

Registers

x: 5
y: 7
z: 12

Just a list of the registers
(they aren't really called x, y, and z)

Has the **ACTUAL** address of the **next** instruction in here

Program Counter

2

Arithmetic Logic Unit

$5 + 7 = 12$

The Simple Language of a MIPS CPU

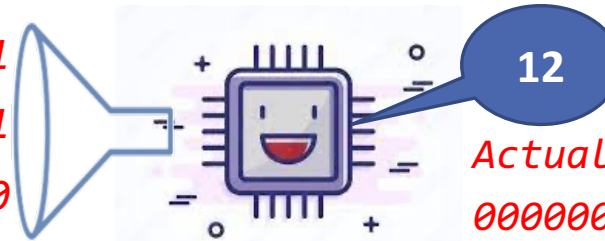
How do we get from: $z = 5 + 7;$

to:

```
addi $t0, $zero, 5    # Assign 5 to register "$t0"  
addi $t1, $zero, 7    # Assign 7 to register "$t1"  
add $t2, $t1, $t0     # Add "$t0" to "$t1" and put the result in reg. "$t2"
```

to:

0010000000001000000000000000101
0010000000001001000000000000111
00000001001010000101000000100000



Actually, it “says”:

00000000000000000000000000001100

That Code Again?

```
addi $t0, $zero, 5  # Assign 5 to "$t0"  
addi $t1, $zero, 7  # Assign 7 to "$t1"  
add  $t2, $t1, $t0   # Add "$t0" to "$t1" and put the result in "$t2"
```

REGISTER:

A data container
inside a CPU.

MIPS Registers
contain 32 bits

\$t0 **\$t1** **\$t2** are called “**Registers**” in the MIPS CPU

- ALL registers each contain **32-bit** values in them – there are 32 such registers in MIPS
- Most registers can be *written to/overwritten* and *read from*

\$zero is also a “**Register**” in the MIPS CPU

- *It's specialized*: it holds a zero value (in 32 bits, naturally) and cannot be overwritten (we can only read it)
- There are other specialized Registers in the MIPS CPU

List of all MIPS Registers

Count: **32**

Size of each one: **32 bits**

Register		Do we use in CS 64?
\$zero	(1)	Y
\$t0 - \$t9	(10)	Y
\$v0, \$v1	(2)	Y
\$a0 - \$a3	(4)	Y
\$s0 - \$s7	(8)	Y
\$sp	(1)	Y
\$ra	(1)	Y
\$k0, \$k1	(2)	N
\$gp	(1)	N
\$fp	(1)	N
\$at	(1)	N

TOTAL: (32)

That Code Again?

```
addi $t0, $zero, 5  # Assign 5 to "$t0"  
addi $t1, $zero, 7  # Assign 7 to "$t1"  
add  $t2, $t1, $t0   # Add "$t0" to "$t1" and put the result in "$t2"
```

“**add**” and “**addi**” are **core instructions** of the MIPS CPU

- There are several of these in MIPS
- They are classified in 3 groups (**R**egister-types, **I**mmEDIATE-types, and **J**ump-types)
- Each group has its own syntax

You can likely guess what these mnemonics mean...

- **add**
- **addi**

List of MIPS Core Instructions

Types (aka Formats): 3 of them (R, I, J)

Size of each full core instruction: 32 bits

CORE INSTRUCTION SET			OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
NAME, MNEMONIC	FOR-MAT			
Add	add	R	$R[rd] = R[rs] + R[rt]$	(1) 0 / 20 _{hex}
Add Immediate	addi	I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu	I	$R[rt] = R[rs] + \text{SignExtImm}$	(2) 9 _{hex}
Add Unsigned	addu	R	$R[rd] = R[rs] + R[rt]$	0 / 21 _{hex}
And	and	R	$R[rd] = R[rs] \& R[rt]$	0 / 24 _{hex}
And Immediate	andi	I	$R[rt] = R[rs] \& \text{ZeroExtImm}$	(3) c _{hex}
Branch On Equal	beq	I	if($R[rs] == R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 4 _{hex}
Branch On Not Equal	bne	I	if($R[rs] != R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 5 _{hex}
Jump	j	J	$PC = \text{JumpAddr}$	(5) 2 _{hex}
Jump And Link	jal	J	$R[31] = PC + 8; PC = \text{JumpAddr}$	(5) 3 _{hex}
Jump Register	jr	R	$PC = R[rs]$	0 / 08 _{hex}
Load Byte Unsigned	lbu	I	$R[rt] = \{24'b0, M[R[rs] + \text{SignExtImm}](7:0)\}$	(2) 24 _{hex}
Load Halfword Unsigned	lhu	I	$R[rt] = \{16'b0, M[R[rs] + \text{SignExtImm}](15:0)\}$	(2) 25 _{hex}
Load Linked	ll	I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2,7) 30 _{hex}
Load Upper Imm.	lui	I	$R[rt] = \{\text{imm}, 16'b0\}$	f _{hex}
Load Word	lw	I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2) 23 _{hex}
Nor	nor	R	$R[rd] = \sim (R[rs] R[rt])$	0 / 27 _{hex}
Or	or	R	$R[rd] = R[rs] R[rt]$	0 / 25 _{hex}
Or Immediate	ori	I	$R[rt] = R[rs] \text{ZeroExtImm}$	(3) d _{hex}
Set Less Than	slt	R	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$	0 / 2a _{hex}
Set Less Than Imm.	slti	I	$R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$	(2) a _{hex}
Set Less Than Imm. Unsigned	sltiu	I	$R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$	(2,6) b _{hex}
Set Less Than Unsig.	sltu	R	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$	(6) 0 / 2b _{hex}
Shift Left Logical	sll	R	$R[rd] = R[rt] \ll \text{shamt}$	0 / 00 _{hex}
Shift Right Logical	srl	R	$R[rd] = R[rt] \gg \text{shamt}$	0 / 02 _{hex}
Store Byte	sb	I	$M[R[rs] + \text{SignExtImm}](7:0) = R[rt](7:0)$	(2) 28 _{hex}
Store Conditional	sc	I	$M[R[rs] + \text{SignExtImm}] = R[rt];$ $R[rt] = (\text{atomic}) ? 1 : 0$	(2,7) 38 _{hex}
Store Halfword	sh	I	$M[R[rs] + \text{SignExtImm}](15:0) = R[rt](15:0)$	(2) 29 _{hex}
Store Word	sw	I	$M[R[rs] + \text{SignExtImm}] = R[rt]$	(2) 2b _{hex}
Subtract	sub	R	$R[rd] = R[rs] - R[rt]$	(1) 0 / 22 _{hex}
Subtract Unsigned	subu	R	$R[rd] = R[rs] - R[rt]$	0 / 23 _{hex}

What's the Rest of this Stuff?

More on that later!

Syntax for **addi** and **add**

```
addi <register1>, <register2>, <immediate value>
```

register1 and **register2** are register names

immediate value is a 16-bit signed integer value

Example1: addi \$t0, \$zero, 5

Example2: addi \$a0, \$a1, -100

```
add <register_d>, <register_s>, <register_t>
```

register_d is a register name (destination)

register_s and **register_t** are register names (source and target)

Example1: add \$t3, \$t0, \$t1

Example2: add \$s0, \$s0, \$s0

That Code Again?

```
addi $t0, $zero, 5  # Assign 5 to "$t0"  
addi $t1, $zero, 7  # Assign 7 to "$t1"  
add $t2, $t1, $t0    # Add "$t0" to "$t1" and put the result in "$t2"
```

ALL *core instructions* in MIPS are **32 bits** in Machine Language

- e.g., the 1st instruction... `addi $t0, $zero, 5`
- ... translates directly to: `0010000000001000000000000000101`
(trust me, there are 32 bits in there!)

Which would you rather read or program with: Machine Language or Assembly??

So Let's Program in Assembly Already!

```
addi $t0, $zero, 5  # Assign 5 to "$t0"  
addi $t1, $zero, 7  # Assign 7 to "$t1"  
add $t2, $t1, $t0    # Add "$t1" to "$t2" and put the result in "$t2"
```

We can **EMULATE** this code on a computer

- That is, we can “pretend” we’re running these instructions on an actual MIPS CPU hardware and “see” what the CPU would do with them
- Using the **spim** program – a MIPS Emulator (learn more about it in this week’s lab)

Can we just type these 3 lines into **spim** and see the result?

- Alas, if only it were that easy...

Running an Emulation of a MIPS Processor

Emulate: v. to reproduce the function of

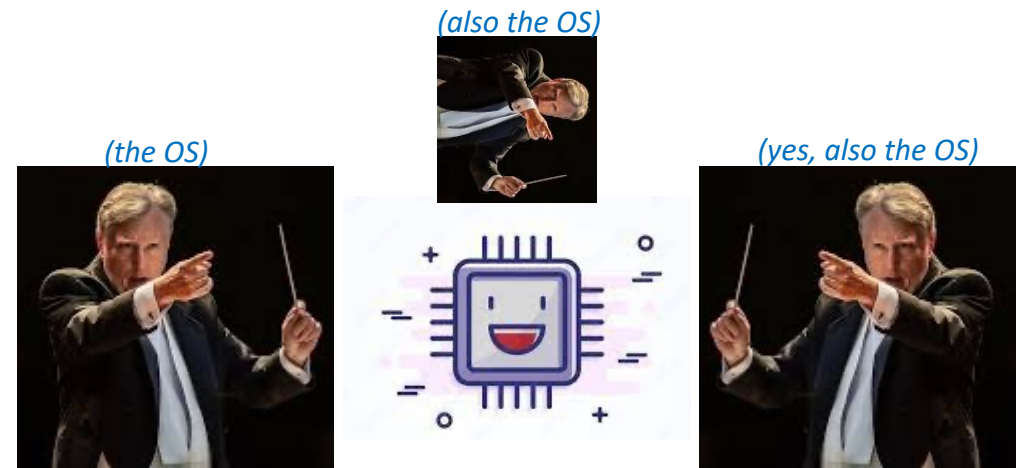
We don't need to mess with electronics and hardware!

We can use **spim** to tell us if/how well this code *would* work if it were on a MIPS CPU

But recall some things that CPUs do NOT do / not in charge of:

- **Loading** a program
- **Starting the Execution** of a program
- Displaying results to **standard output**
- Get values from **standard input**

What does this in a computer system?



Emulating and Assessing Properly

1. SET UP THE ENVIRONMENT FOR THE EMULATOR

2. WRITE THE ACTUAL INSTRUCTIONS

```
addi $t0, $zero, 5    # Assign 5 to "$t0"
```

```
addi $t1, $zero, 7    # Assign 7 to "$t1"
```

```
add $t2, $t1, $t0     # Add "$t0" to "$t1" and put the result in "$t2"
```

3. SHOW (print to std.out) THE RESULT BY EMULATING THE EXCHANGE BETWEEN CPU and OS

4. QUIT THE PROGRAM "PROPERLY"

```
.text    # We always have to have this starting line - this is called a directive
main:   # "main:" is another label - it is also necessary (label ≠ CPU instruction)

    addi $t0, $zero, 5
    addi $t1, $zero, 7
    add  $t2, $t0, $t1

# Print the integer that's in $t2 to std.output - MORE ON HOW THIS WORKS LATER!

    li $v0, 1
    move $a0, $t2
    syscall

# End program - MORE ON HOW THIS WORKS LATER!

    li $v0, 10
    syscall
```

Look for the demo file:
"simpleAdd.asm"

Wow... All of this code just so
we can add 5 to 7
and display the result!!!

The MIPS CPU and the “*Magical*” Number 32

REGISTERS:

There are 32 registers and each register holds 32 bits

INSTRUCTIONS:

All MIPS instructions are 32 bits long

CPU MEMORY:

MIPS CPU memory uses 32 bits for its addresses

- That means there's 2^{32} total addressable memory – that's equivalent to *about* 4 Billion addresses
- How much memory is that??? *Weeeellll.... more on that next week...!*

YOUR TO-DOS

Go to lab sections on Friday

Read ALL of **Handout 2** (it's on Canvas)

Watch the video! (it's on Canvas)

Turn in **homework 2** on Monday by 11:59 pm

Come to office hours!

</LECTURE>